JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Interface Enumeration<E>

All Known Subinterfaces:

NamingEnumeration<T>

All Known Implementing Classes:

StringTokenizer

```
public interface Enumeration<E>
```

An object that implements the Enumeration interface generates a series of elements, one at a time. Successive calls to the `nextElement` method return successive elements of the series.

For example, to print all elements of a `Vector<E>` *v*:

```
   for (Enumeration<E> e = v.elements(); e.hasMoreElements();)
       System.out.println(e.nextElement());
```

Methods are provided to enumerate through the elements of a vector, the keys of a hashtable, and the values in a hashtable. Enumerations are also used to specify the input streams to a `SequenceInputStream`.

NOTE: The functionality of this interface is duplicated by the Iterator interface. In addition, Iterator adds an optional remove operation, and has shorter method names. New implementations should consider using Iterator in preference to Enumeration.

Since:

JDK1.0

See Also:

Iterator, SequenceInputStream, nextElement(), Hashtable, Hashtable.elements(), Hashtable.keys(), Vector, Vector.elements()

Method Summary

Abstract Methods

Modifier and Type	Method	Description
boolean	hasMoreElements()	Tests if this enumeration contains more elements.
E	nextElement()	Returns the next element of this enumeration if this enumeration object has at least one more element to provide.

Method Detail

hasMoreElements

```
boolean hasMoreElements()
```

Tests if this enumeration contains more elements.

Returns:

`true if and only if this enumeration object contains at least one more element to provide; false otherwise.`

nextElement

```
E nextElement()
```

Returns the next element of this enumeration if this enumeration object has at least one more element to provide.

Returns:

`the next element of this enumeration.`

Throws:

`NoSuchElementException` - if no more elements exist.

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.